Capstone Project Report: KIT (Keep In Touch) Relationship Manager

Toby Pike
Durham College

CPGA 3200 – Capstone Project

April 2026

Introduction

For my capstone project, I built a mobile application called KIT (Keep In Touch), which is a relationship manager. The idea behind it was quite simple. People lose track of when they last talked to someone, and relationships slowly fall off because of that. The app solves that by tracking interactions and showing who you should reach out to and when.

I chose this project because I wanted something that forced me to actually build a full system, but also something that I wanted to use myself so that when designing it I could think of features a user would really want. A lot of earlier assignments focused on one area at a time. This time I wanted frontend, backend, database, authentication, and real data flow all working together in one project. KIT ended up being a Flutter mobile app connected to a Node.js and Express API, with Prisma handling the database layer.

Project Work and Contributions

This was an individual project, so everything from architecture to implementation was on me. On the backend, I built a REST API using Node.js and Express running on port 4000. The API handles authentication, user data, people records, interactions, and reminders. I implemented JWT authentication so users can register, log in, and then access protected routes using a bearer token. I verified all of this through Postman before ever connecting it to the frontend, which saved a lot of time later.

The database is managed through Prisma using SQLite in development. I designed three main models: User, Person, and Interaction. A user can create multiple people, and each person can have multiple interactions. Each person also has a check-in interval, which is used to calculate when they are “due” for contact. That logic is handled through a reminders endpoint that returns two lists: people who are due now and people who are coming up soon. That ended up being one of the more important features.

On the frontend, I built the mobile app using Flutter. I connected it to the backend using the emulator base URL 10.0.2.2:4000, which maps to localhost. The app includes a login screen, dashboard, and forms for adding people and logging interactions. Once a user logs in, the token is stored and used for all future API requests.

The dashboard is where most of the value is. It shows a “Due Now” section and an “Upcoming” section based on the reminder logic from the backend. It also shows the last time you contacted someone, which updates every time an interaction is logged. This required syncing frontend state with backend data properly, which took some trial and error.

A lot of the work was making sure everything connects properly. Authentication, API calls, database updates, and UI all had to line up. When something broke, it usually broke across multiple layers, so debugging meant tracing the issue from the frontend all the way back to the database.

Approach and Commitment

I built this project in stages over the semester. First, I focused on getting the backend working. I made sure I could register a user, log in, and hit protected routes with a token. Once that was stable, I added the core data models and CRUD operations for people and interactions.

After that, I connected the Flutter frontend. At that point, most of the work shifted to integration. Getting requests to work from the emulator, handling tokens properly, and making sure data updates were reflected in the UI. One of the more frustrating parts was authentication. There were times where the API worked in Postman but failed in the app because of how headers or tokens were being handled, and fixing it forced me to understand how the full request flow works.

I also spent a lot of time testing small pieces before moving on. Instead of building everything and hoping it works, I made sure each endpoint worked, then each screen, then the connection between them. It might have made certain parts slower to build but it made me understand the system better. A lot of time went into fixing bugs and cleaning things up instead of adding new features.

Reflection and Learning

This project felt completely different from anything earlier in the program. Usually there is a task to accomplish, even if it is up to me how to approach it. This project was different because it required me to come up with and fully develop my own idea.

One of the biggest realizations was how important structure is early on. There were points where I rushed into building something, and it worked short term but caused long term issues that required fixes. Going back and fixing those decisions made it clear that planning matters more than I gave it credit for.

Another big takeaway was debugging. A lot of this project was not writing new code but figuring out why something was not working. This included API errors, incorrect data handling, token issues, and emulator networking problems that required debugging across multiple layers of the system.

The moment things really clicked was when the full flow worked end to end. Logging in, creating a person, logging an interaction, and then seeing that person move into the “Due Now” section based on the logic I wrote. That was the first time it actually felt like what I was thinking of in my head had come into reality.

I also learned how important it is to separate responsibilities. The backend handles logic and data, the frontend handles display, and Prisma sits in between. Keeping those roles clear made things easier to manage as the project grew.

If I did this again, I would plan the data model and feature scope more upfront. I changed things a few times during development, which worked in the end, but cost time.

Capstone Justification

This project represents what I have learned in the program because it combines multiple areas into one system. It includes backend development with Node.js and Express, API design and testing, authentication using JWT, database design and relationships using Prisma, frontend development with Flutter, and real data flow between all layers.

More importantly, it shows that I can take an idea and turn it into a working application. This includes writing code, debugging issues, making design decisions, and continuously improving the system.

The project is also based on a real problem. It is something that could be expanded into a real product, and not just something built to meet assignment requirements. That makes it a strong example of practical development.

This project shows that I can work through problems independently, connect multiple technologies into a single system, and deliver a working product.

Conclusion

KIT was a good choice for a capstone project because it pushed me into areas I had not fully dealt with before, especially full integration between the frontend and backend, as well as mobile application development.

There were definitely parts that could be improved with more time. I would have added things like push notifications and worked to have better UI design. But the core system works and shows a solid foundation.

Looking back, I am happy with how the project turned out. It feels like something that represents what I can do now. More than anything, this project showed me that I can take an idea, build it into a complete system, and work through the challenges that come with real development.